

Déploiement en Production - JobbingTrack

Ce guide explique comment déployer JobbingTrack en mode production sur votre serveur personnel avec **Portainer** et **Nginx Proxy Manager**.

Prérequis

-  Serveur Linux (Ubuntu/Debian recommandé)
-  Docker et Docker Compose installés
-  Portainer installé et configuré
-  Nginx Proxy Manager installé et configuré
-  Domaine pointant vers votre serveur
-  Certificat SSL (Let's Encrypt automatique via NPM)

Structure de Déploiement

Fichiers à Préparer

```
production/
├─ docker-compose.yml      # Stack principale
├─ .env.production         # Variables d'environnement
├─ nginx/
│   └─ proxy-hosts/        # Configurations Nginx personnalisées
├─ init-scripts/
│   ├── init.sql           # Initialisation base de données
│   └─ setup-admin.sh      # Création utilisateur admin
├─ backups/
│   └─ scripts/            # Scripts de sauvegarde
└─ monitoring/
    ├── prometheus.yml     # Configuration Prometheus
    └─ grafana-dashboards/ # Dashboards personnalisés
```

Préparation des Fichiers

Étape 1 : Créer le Répertoire de Production

```
# Sur votre serveur
mkdir -p /opt/jobbingtrack-production
cd /opt/jobbingtrack-production

# Structure complète
mkdir -p nginx/proxy-hosts init-scripts backups/scripts monitoring/grafana-dashboa
```

Étape 2 : Fichiers Docker Compose de Production

`docker-compose.yml` (Principal)

```
version: '3.8'

services:
  # Base de données PostgreSQL
  postgres:
    image: postgres:15-alpine
    container_name: jobbingtrack-postgres-prod
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts/init.sql:/docker-entrypoint-initdb.d/01-init.sql
    networks:
      - jobbingtrack-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 30s
      timeout: 10s
      retries: 5

  # Redis pour le cache et les sessions
  redis:
    image: redis:7-alpine
    container_name: jobbingtrack-redis-prod
    restart: unless-stopped
    command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    networks:
      - jobbingtrack-network
    healthcheck:
      test: ["CMD", "redis-cli", "--raw", "incr", "ping"]
      interval: 30s
```

```

    timeout: 10s
    retries: 5

# API Gateway (port interne)
api-gateway:
  image: votre-registry/jobbingtrack-api-gateway:latest
  container_name: jobbingtrack-api-gateway-prod
  restart: unless-stopped
  environment:
    NODE_ENV: production
    PORT: 3000
    DOCKER_STATS_SERVICE_URL: http://docker-stats:3015
    AUTH_SERVICE_URL: http://auth-service:3001
    APPLICATION_SERVICE_URL: http://application-service:3002
    COMPANY_SERVICE_URL: http://company-service:3003
    CONTACT_SERVICE_URL: http://contact-service:3004
    INTERVIEW_SERVICE_URL: http://interview-service:3005
    NOTIFICATION_SERVICE_URL: http://notification-service:3006
    DASHBOARD_SERVICE_URL: http://dashboard-service:3007
    CALL_SERVICE_URL: http://call-service:3008
    PROFILE_SERVICE_URL: http://profile-service:3009
    EVENT_SERVICE_URL: http://event-service:3011
    FOLLOWUP_SERVICE_URL: http://followup-service:3012
    WORKFLOW_SERVICE_URL: http://workflow-service:3013
    ALLOWED_ORIGINS: https://${DOMAIN},https://www.${DOMAIN}
    JWT_SECRET: ${JWT_SECRET}
    DOCKER_SOCKET_PATH: /var/run/docker.sock
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - docker_stats:/app/data
  networks:
    - jobbingtrack-network
  depends_on:
    - postgres
    - redis
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.jobbingtrack.rule=Host(`${DOMAIN}`)"
    - "traefik.http.routers.jobbingtrack.entrypoints=websecure"
    - "traefik.http.routers.jobbingtrack.tls.certresolver=letsencrypt"
    - "traefik.http.services.jobbingtrack.loadbalancer.server.port=3000"

# Services Microservices (identique à développement mais optimisé)
auth-service:
  image: votre-registry/jobbingtrack-auth-service:latest
  container_name: jobbingtrack-auth-service-prod
  restart: unless-stopped
  environment:
    NODE_ENV: production
    PORT: 3001
    DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
    JWT_SECRET: ${JWT_SECRET}

```

```

JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
FRONTEND_URL: https://${DOMAIN}
API_URL: https://${DOMAIN}
networks:
  - jobbingtrack-network
depends_on:
  postgres:
    condition: service_healthy

# Ajouter les autres services ici (application-service, etc.)
# ... (copier depuis le docker-compose.yml de développement)

# Service de statistiques Docker (optionnel en prod)
docker-stats-service:
  image: votre-registry/jobbingtrack-docker-stats:latest
  container_name: jobbingtrack-docker-stats-prod
  restart: unless-stopped
  environment:
    NODE_ENV: production
    PORT: 3015
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
  networks:
    - jobbingtrack-network

# Monitoring avec Prometheus
prometheus:
  image: prom/prometheus:latest
  container_name: jobbingtrack-prometheus-prod
  restart: unless-stopped
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/etc/prometheus/console_libraries'
    - '--web.console.templates=/etc/prometheus/consoles'
    - '--storage.tsdb.retention.time=200h'
    - '--web.enable-lifecycle'
  networks:
    - jobbingtrack-network
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.prometheus.rule=Host(`${DOMAIN}`)"
    - "traefik.http.routers.prometheus.entrypoints=websecure"
    - "traefik.http.routers.prometheus.tls.certresolver=letsencrypt"

# Grafana pour les dashboards
grafana:
  image: grafana/grafana:latest
  container_name: jobbingtrack-grafana-prod

```

```

restart: unless-stopped
environment:
  GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
  GF_USERS_ALLOW_SIGN_UP: "false"
volumes:
  - grafana_data:/var/lib/grafana
  - ./monitoring/grafana-dashboards:/var/lib/grafana/dashboards
networks:
  - jobbingtrack-network
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.grafana.rule=Host(`grafana.${DOMAIN}`)"
  - "traefik.http.routers.grafana.entrypoints=websecure"
  - "traefik.http.routers.grafana.tls.certresolver=letsencrypt"

volumes:
  postgres_data:
    driver: local
  redis_data:
    driver: local
  prometheus_data:
    driver: local
  grafana_data:
    driver: local
  docker_stats:
    driver: local

networks:
  jobbingtrack-network:
    driver: bridge
    name: jobbingtrack-prod

```

init-scripts/init.sql

```

-- Initialisation base de données production
-- Ce fichier est exécuté automatiquement au premier lancement

-- Extensions utiles
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Configuration optimisée pour la production
ALTER DATABASE jobbingtrack SET timezone = 'UTC';
ALTER DATABASE jobbingtrack SET log_statement = 'ddl';
ALTER DATABASE jobbingtrack SET log_min_duration_statement = 1000;

```

```
-- Création des schémas si nécessaire
-- Les migrations Prisma seront appliquées après
```

Étape 3 : Variables d'Environnement de Production

`.env.production`

```
# =====
# VARIABLES D'ENVIRONNEMENT - PRODUCTION
# =====
# ⚠ IMPORTANT : Ne jamais committer ce fichier !

# Configuration Base de Données
DB_NAME=jobbingtrack_prod
DB_USER=jobbingtrack_user
DB_PASSWORD=VOTRE_MOT_DE_PASSE_COMPLEXE_ICI
POSTGRES_PASSWORD=MÊME_MOT_DE_PASSE_QUE_DB_PASSWORD

# Configuration Redis
REDIS_PASSWORD=VOTRE_MOT_DE_PASSE_REDIS_COMPLEXE

# Configuration JWT (générer des clés aléatoires)
JWT_SECRET=VOTRE_JWT_SECRET_TRES_LONG_ET_COMPLEXE_256_BITS_MINIMUM
JWT_REFRESH_SECRET=VOTRE_JWT_REFRESH_SECRET_DIFFERENT_TRES_LONG

# Configuration Domaine
DOMAIN=votre-domaine.com

# Configuration Email (votre serveur SMTP)
SMTP_HOST=votre-serveur-smtp.com
SMTP_PORT=587
SMTP_USER=votre-email@domaine.com
SMTP_PASS=votre-mot-de-passe-smtp
SMTP_FROM=JobbingTrack <votre-email@domaine.com>

# Configuration Monitoring
GRAFANA_PASSWORD=VOTRE_MOT_DE_PASSE_GRAFANA_COMPLEXE

# Configuration Sécurité
NODE_ENV=production
COMPOSE_PROJECT_NAME=jobbingtrack-prod

# Configuration Logs
LOG_LEVEL=info
LOG_FORMAT=json

# Configuration Backup
BACKUP_RETENTION_DAYS=30
```

```
BACKUP_TIME=02:00
```

```
# =====
# GÉNÉRATION DES MOTS DE PASSE SÉCURISÉS
# =====

# Utilisez ces commandes pour générer des mots de passe sécurisés :
# openssl rand -hex 32 # Pour JWT secrets
# openssl rand -hex 16 # Pour mots de passe base de données
# pwgen -s 20 -1      # Pour mots de passe alphanumériques
```

2 Configuration Nginx Proxy Manager

Étape 1 : Ajouter votre Domaine

1. Connectez-vous à Nginx Proxy Manager

- URL : `http://IP_SERVEUR:81`
- Identifiants par défaut : `admin@example.com` / `changeme`

2. Ajouter un Proxy Host

- **Domain Names** : `jobbingtrack.votre-domaine.com` (et `www.jobbingtrack.votre-domaine.com`)
- **Scheme** : `http`
- **Forward Host** : `IP_DU_SERVEUR`
- **Forward Port** : `3000` (port de l'API Gateway)
- **SSL** : ☒ Activer
- **Force SSL** : ☒
- **HTTP/2 Support** : ☒
- **Certificate** : Let's Encrypt

3. Configuration Avancée

```
# Ajouter ces lignes dans "Custom Nginx Configuration"
location / {
    proxy_pass http://localhost:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Configuration pour les WebSockets (si utilisés)
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
```

```
# Timeout pour éviter les déconnexions
proxy_read_timeout 86400;
proxy_send_timeout 86400;
}

# Configuration pour les fichiers statiques (si nécessaire)
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

Étape 2 : Sous-domaines pour Monitoring

1. **Grafana** : `grafana.votre-domaine.com` → Port 3001
2. **Prometheus** : `prometheus.votre-domaine.com` → Port 9090

3 Script de Déploiement Automatisé

`deploy.sh`

```
#!/bin/bash

# =====
# Script de Déploiement Automatisé - JobbingTrack Production
# =====

set -e

# Couleurs pour les messages
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BLUE='\033[0;34m'
NC='\033[0m'

# Configuration
PROJECT_NAME="jobbingtrack-production"
PROJECT_DIR="/opt/${PROJECT_NAME}"
BACKUP_DIR="${PROJECT_DIR}/backups"
LOG_FILE="${PROJECT_DIR}/deploy-$(date +%Y%m%d-%H%M%S).log"

# Fonction de logging
log() {
    echo -e "${BLUE}[$(date '+%Y-%m-%d %H:%M:%S')${NC}] $1" | tee -a "$LOG_FILE"
}

# Fonction d'erreur
```



```

error() {
    echo -e "${RED}[ERREUR}${NC} $1" | tee -a "$LOG_FILE"
    exit 1
}

# Fonction de succès
success() {
    echo -e "${GREEN}[OK}${NC} $1" | tee -a "$LOG_FILE"
}

# =====
# VÉRIFICATIONS PRÉLIMINAIRES
# =====

log "🚀 Démarrage du déploiement de JobbingTrack en production"

# Vérifier les outils nécessaires
command -v docker >/dev/null 2>&1 || error "Docker n'est pas installé"
command -v docker-compose >/dev/null 2>&1 || error "Docker Compose n'est pas installé"

# Vérifier le fichier d'environnement
if [ ! -f ".env.production" ]; then
    error "Fichier .env.production manquant"
fi

# Charger les variables d'environnement
log "📋 Chargement des variables d'environnement..."
source .env.production

# =====
# SAUVEGARDE
# =====

log "💾 Création d'une sauvegarde avant déploiement..."

# Créer le répertoire de sauvegarde
mkdir -p "${BACKUP_DIR}/${date +%Y%m%d}"

# Sauvegarder la base de données si elle existe
if docker ps -q -f name=jobbingtrack-postgres-prod >/dev/null 2>&1; then
    log "📦 Sauvegarde de la base de données..."
    docker exec jobbingtrack-postgres-prod pg_dump \
        -U ${DB_USER} \
        -h localhost \
        ${DB_NAME} > "${BACKUP_DIR}/${date +%Y%m%d}/db-backup.sql"
    success "Base de données sauvegardée"
fi

# =====
# DÉPLOIEMENT
# =====

```

```

log "🏗️ Déploiement de l'infrastructure..."

# Arrêter les services existants
log "🛑 Arrêt des services existants..."
docker-compose down 2>/dev/null || true

# Créer les réseaux si nécessaire
log "🌐 Création du réseau Docker..."
docker network create jobbingtrack-prod 2>/dev/null || true

# Démarrer l'infrastructure de base
log "🚀 Démarrage de PostgreSQL et Redis..."
docker-compose up -d postgres redis

# Attendre que PostgreSQL soit prêt
log "⌚ Attente de PostgreSQL..."
sleep 30

# Vérifier que PostgreSQL répond
if ! docker exec jobbingtrack-postgres-prod pg_isready -U ${DB_USER} -d ${DB_NAME}
    error "PostgreSQL ne répond pas"
fi

# Appliquer les migrations
log "📦 Application des migrations Prisma..."
docker-compose exec -T auth-service npx prisma migrate deploy
docker-compose exec -T auth-service npx prisma generate

# Créer l'utilisateur administrateur
log "👤 Création de l'utilisateur administrateur..."
docker-compose exec -T postgres psql -U ${DB_USER} -d ${DB_NAME} -c "
INSERT INTO \"User\" (id, email, password, \"firstName\", \"lastName\", role, \"is
VALUES (
    'admin_$(date +%s)',
    'admin@${DOMAIN}',
    '\$2b\$10\$92IXUNpkj00rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',
    'Super',
    'Administrateur',
    'SUPER_ADMIN',
    true,
    NOW(),
    NOW()
)
ON CONFLICT (email) DO UPDATE SET
    role = 'SUPER_ADMIN',
    \"isActive\" = true,
    \"updatedAt\" = NOW()
RETURNING id, email, role;"

# Démarrer tous les services
log "🌟 Démarrage de tous les services..."
docker-compose up -d

```

```

# Attendre que les services soient prêts
log "🕒 Attente du démarrage complet..."
sleep 60

# =====
# VÉRIFICATIONS POST-DÉPLOIEMENT
# =====

log "🔍 Vérifications post-déploiement..."

# Vérifier la santé des services
SERVICES=("api-gateway" "auth-service" "application-service")
for service in "${SERVICES[@]}; do
    if curl -f -s http://localhost:3000/health >/dev/null 2>&1; then
        success "Service $service répond correctement"
    else
        error "Service $service ne répond pas"
    fi
done

# Vérifier l'accès externe
if curl -f -s -k https://{DOMAIN}/health >/dev/null 2>&1; then
    success "Application accessible via le domaine"
else
    error "Application non accessible via le domaine"
fi

# =====
# CONFIGURATION SSL
# =====

log "🔒 Vérification du certificat SSL..."
# Nginx Proxy Manager gère automatiquement Let's Encrypt

# =====
# MONITORING
# =====

log "📊 Démarrage du monitoring..."
# Prometheus et Grafana démarrent automatiquement avec docker-compose

# =====
# FINALISATION
# =====

success "🎉 Déploiement terminé avec succès !"

echo ""
echo -e "${GREEN}🌐 Accès à l'application :${NC}"
echo "  Frontend:      https://{DOMAIN}"
echo "  API Gateway:   https://{DOMAIN}"

```

```

echo ""
echo -e "${BLUE}📊 Monitoring :${NC}"
echo "   Grafana:      https://grafana.${DOMAIN}"
echo "   Prometheus:    https://prometheus.${DOMAIN}"
echo ""
echo -e "${YELLOW}🔑 Identifiants administrateur :${NC}"
echo "   Email:         admin@${DOMAIN}"
echo "   Mot de passe: SuperAdmin123!"
echo ""
echo -e "${RED}⚠️ IMPORTANT :${NC}"
echo "   - Changez immédiatement le mot de passe administrateur"
echo "   - Configurez les sauvegardes automatiques"
echo "   - Activez la surveillance des logs"
echo "   - Testez toutes les fonctionnalités"
echo ""
echo -e "${BLUE}📋 Prochaines étapes :${NC}"
echo "   1. Tester l'interface web"
echo "   2. Configurer les notifications email"
echo "   3. Programmer les sauvegardes"
echo "   4. Configurer la surveillance"

log "✅ Déploiement terminé - Voir $LOG_FILE pour les détails"

```

4 Procédure de Déploiement Étape par Étape

Étape 1 : Préparation sur votre Machine Locale

```

# 1. Construire les images Docker
docker-compose build --parallel

# 2. Tester en local
make up
# Vérifier que tout fonctionne

# 3. Arrêter les services locaux
make down

# 4. Préparer les fichiers de production
cp docker-compose.yml production/
cp .env.production production/
# Ajouter les autres fichiers nécessaires

# 5. Construire les images pour votre registry
docker tag votre-image-api-gateway votre-registry/jobbingtrack-api-gateway:latest
docker tag votre-image-auth-service votre-registry/jobbingtrack-auth-service:latest
# ... pour tous les services

```

```
# 6. Pousser vers votre registry
docker push votre-registry/jobbingtrack-api-gateway:latest
# ... pour toutes les images
```

Étape 2 : Déploiement sur le Serveur

Méthode 1 : Via Portainer (Recommandé)

1. Créer un Stack dans Portainer

- Aller dans **Stacks** → **Add stack**
- **Name** : `jobbingtrack-production`
- **Build method** : `Web editor`
- Copier le contenu du `docker-compose.yml` de production

2. Variables d'Environnement

- Aller dans **Environment** du stack
- Ajouter toutes les variables du `.env.production`

3. Déployer

- Cliquer sur **Deploy the stack**
- Surveiller les logs de déploiement

Méthode 2 : Manuel via SSH

```
# Sur votre serveur
cd /opt/jobbingtrack-production

# Télécharger/mettre à jour les fichiers
# (git pull, scp, rsync, etc.)

# Rendre le script exécutable
chmod +x deploy.sh

# Exécuter le déploiement
./deploy.sh
```

Étape 3 : Configuration Nginx Proxy Manager

1. Ajouter le Proxy Host Principal

- Domain : `jobbingtrack.votre-domaine.com`
- Forward IP : `127.0.0.1`
- Forward Port : `3000`

2. Sous-domaines Monitoring

- `grafana.votre-domaine.com` → Port 3001
- `prometheus.votre-domaine.com` → Port 9090

3. Configuration SSL

- Activer **SSL Certificate**
- Sélectionner **Request a new SSL Certificate (Let's Encrypt)**
- Accepter les termes de service

5 Monitoring et Maintenance

Configuration Prometheus (`monitoring/prometheus.yml`)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'jobbingtrack-services'
    static_configs:
      - targets: ['api-gateway:3000', 'auth-service:3001', 'application-service:3002']
    scrape_interval: 30s

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

Sauvegardes Automatisées

Script de Sauvegarde (`backups/scripts/backup.sh`)

```
#!/bin/bash
# Script de sauvegarde quotidien

BACKUP_DIR="/opt/jobbingtrack-production/backups/$(date +%Y%m%d)"
```

```
mkdir -p "$BACKUP_DIR"

# Sauvegarde base de données
docker exec jobbingtrack-postgres-prod pg_dump \
  -U $DB_USER \
  -h localhost \
  $DB_NAME > "$BACKUP_DIR/db-backup.sql"

# Sauvegarde volumes Docker
docker run --rm \
  -v jobbingtrack_postgres_data:/data \
  -v "$BACKUP_DIR:/backup" \
  alpine tar czf /backup/postgres-data.tar.gz /data

# Nettoyer les anciennes sauvegardes (>30 jours)
find /opt/jobbingtrack-production/backups -type d -mtime +30 -exec rm -rf {} +
```

6 Sécurité et Bonnes Pratiques

Sécurité

1. Mots de Passe Forts

```
# Générer des mots de passe sécurisés
openssl rand -hex 32
```

2. Pare-feu

```
# Autoriser seulement les ports nécessaires
ufw allow 80      # HTTP
ufw allow 443     # HTTPS
ufw allow 81      # Nginx Proxy Manager
```

3. Mises à Jour Automatiques

```
# Configuration automatique des mises à jour
unattended-upgrades
```

Logs et Monitoring

1. Logs Centralisés

```
# Ajouter dans docker-compose.yml
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
```

2. Alertes

- Configuration Grafana pour les alertes email
- Surveillance des métriques clés (CPU, mémoire, disques)

Maintenance

1. Mises à Jour

```
# Script de mise à jour
cd /opt/jobbingtrack-production
./deploy.sh
```

2. Sauvegardes

```
# Sauvegarde manuelle
./backups/scripts/backup.sh

# Vérification des sauvegardes
ls -la backups/
```

Dépannage

Problèmes Courants

Application Non Accessible

```
# Vérifier les logs
docker-compose logs api-gateway

# Vérifier la connectivité réseau
```



```
curl http://localhost:3000/health
```

```
# Vérifier Nginx Proxy Manager
docker logs nginx-proxy-manager
```

✗ Base de Données Inaccessible

```
# Vérifier PostgreSQL
docker logs jobbingtrack-postgres-prod

# Vérifier la connectivité
docker exec jobbingtrack-postgres-prod pg_isready -U $DB_USER
```

✗ Certificat SSL

```
# Vérifier le statut du certificat
# Dans Nginx Proxy Manager : SSL Certificates

# Renouveler manuellement si nécessaire
docker exec nginx-proxy-manager /app/letsencrypt/renew.sh
```

🎯 Résultat Final

Après déploiement réussi, vous aurez :

-  **Application accessible** : <https://jobbingtrack.votre-domaine.com>
-  **Certificat SSL automatique** via Let's Encrypt
-  **Monitoring complet** avec Grafana et Prometheus
-  **Sauvegardes automatisées** de la base de données
-  **Interface d'administration** pour gérer les services
-  **Logs centralisés** et surveillance

🚀 Commandes Utiles

```
# Gestion des services
docker-compose up -d      # Démarrer
docker-compose down       # Arrêter
docker-compose logs -f    # Logs temps réel
docker-compose ps         # Statut des services
```

```
# Maintenance
./deploy.sh                # Redéployer
./backups/scripts/backup.sh # Sauvegarder
docker system prune -f     # Nettoyer Docker

# Monitoring
curl https://grafana.votre-domaine.com
curl https://prometheus.votre-domaine.com
```

🎉 **Votre JobbingTrack est maintenant déployé en production !**

Navigation

-  [Index](#)
-  [Accueil](#)